

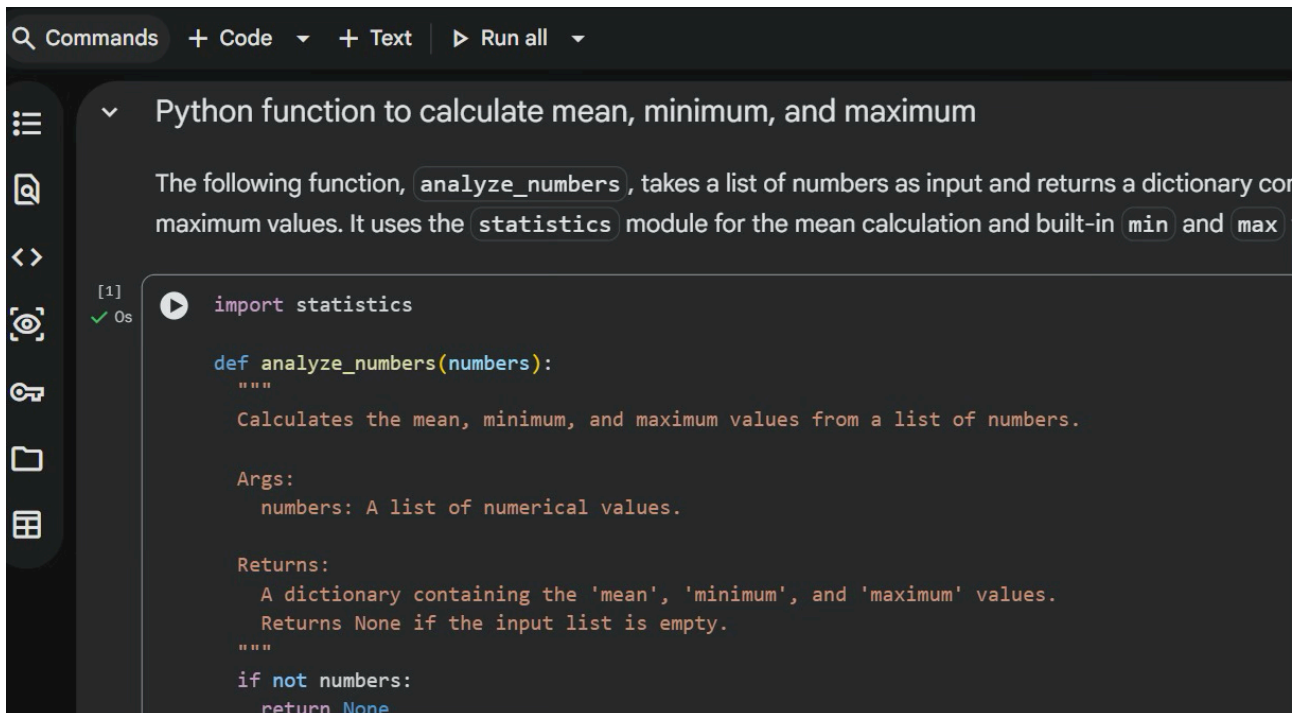
# AI-assisted coding

## 2303A510A0( Vishnu Vardhan )

### Assignment-2.1

#### Task 1:

Use Google Gemini in Colab to generate a Python function that reads a list of numbers and calculates the mean, minimum, and maximum values.



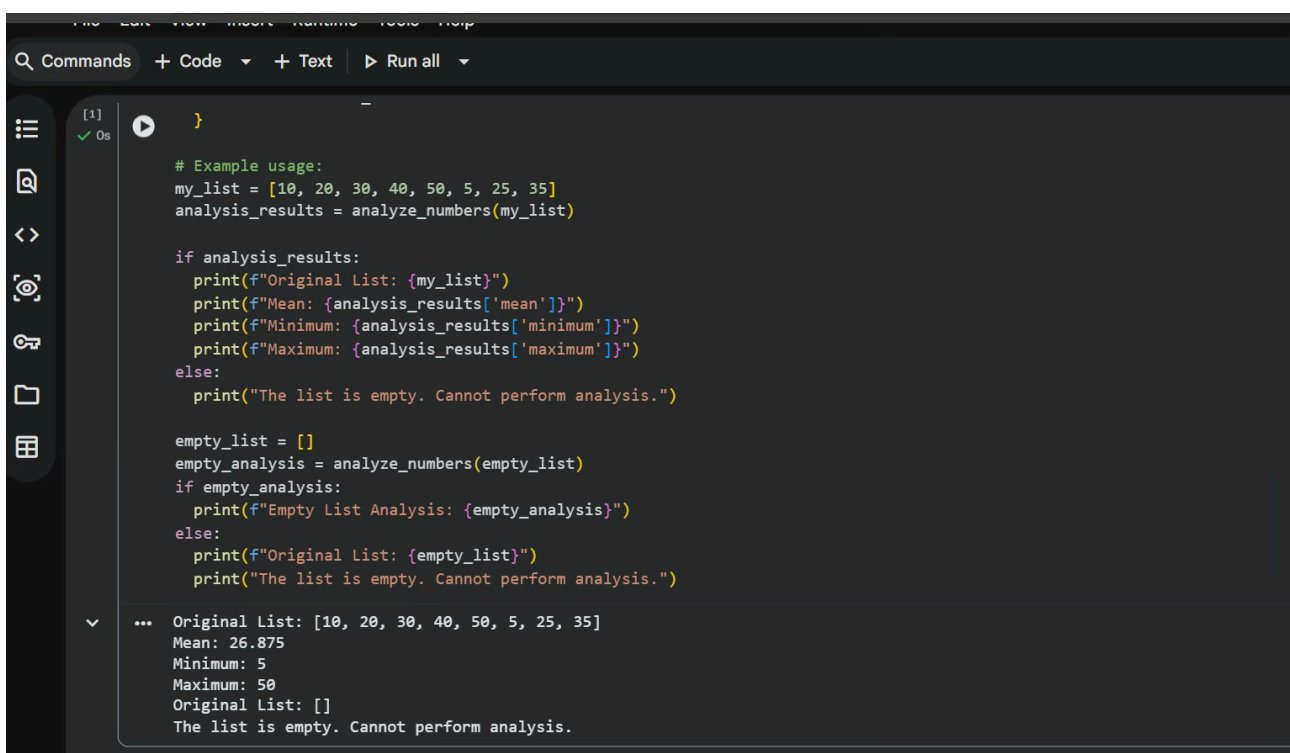
The screenshot shows the Google Colab interface with a code cell titled "Python function to calculate mean, minimum, and maximum". The cell contains the following Python code:

```
[1] ✓ 0s
import statistics

def analyze_numbers(numbers):
    """
    Calculates the mean, minimum, and maximum values from a list of numbers.

    Args:
        numbers: A list of numerical values.

    Returns:
        A dictionary containing the 'mean', 'minimum', and 'maximum' values.
        Returns None if the input list is empty.
    """
    if not numbers:
        return None
```



The screenshot shows the Google Colab interface with a code cell containing the following Python code:

```
[1] ✓ 0s
}

# Example usage:
my_list = [10, 20, 30, 40, 50, 5, 25, 35]
analysis_results = analyze_numbers(my_list)

if analysis_results:
    print(f"Original List: {my_list}")
    print(f"Mean: {analysis_results['mean']}")
    print(f"Minimum: {analysis_results['minimum']}")
    print(f"Maximum: {analysis_results['maximum']}")
else:
    print("The list is empty. Cannot perform analysis.")

empty_list = []
empty_analysis = analyze_numbers(empty_list)
if empty_analysis:
    print(f"Empty List Analysis: {empty_analysis}")
else:
    print(f"Original List: {empty_list}")
    print("The list is empty. Cannot perform analysis.")
```

The output of the code cell is displayed below the code:

```
... Original List: [10, 20, 30, 40, 50, 5, 25, 35]
Mean: 26.875
Minimum: 5
Maximum: 50
Original List: []
The list is empty. Cannot perform analysis.
```

## Task 2 :

## Generate an Armstrong number checker using Gemini and GitHub Copilot.

### Gemini:

```
def is_armstrong_number(number):
    """
    Checks if a given number is an Armstrong number.

    An Armstrong number (also known as a narcissistic number,
    pluperfect digital invariant, or plus perfect number)
    is a number that is the sum of its own digits each raised
    to the power of the number of digits.

    Args:
        number: An integer to check.

    Returns:
        True if the number is an Armstrong number, False otherwise.
    """
    if not isinstance(number, int) or number < 0:
        return False # Armstrong numbers are typically non-negative integers

    num_str = str(number)
    num_digits = len(num_str)
    sum_of_powers = 0

    for digit_char in num_str:
        digit = int(digit_char)
        sum_of_powers += digit ** num_digits

    return sum_of_powers == number

# Example Usage:
print(f"Is 153 an Armstrong number? {is_armstrong_number(153)}") # Expected: True (1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153)
print(f"Is 9 an Armstrong number? {is_armstrong_number(9)}") # Expected: True (9^1 = 9)
print(f"Is 370 an Armstrong number? {is_armstrong_number(370)}") # Expected: True (3^3 + 7^3 + 0^3 = 27 + 343 + 0 = 370)
print(f"Is 371 an Armstrong number? {is_armstrong_number(371)}") # Expected: True (3^3 + 7^3 + 1^3 = 27 + 343 + 1 = 371)
print(f"Is 1634 an Armstrong number? {is_armstrong_number(1634)}") # Expected: True (1^4 + 6^4 + 3^4 + 4^4 = 1 + 1296 + 81 + 256 = 1634)
print(f"Is 123 an Armstrong number? {is_armstrong_number(123)}") # Expected: False
print(f"Is -153 an Armstrong number? {is_armstrong_number(-153)}") # Expected: False
```

```
ASS2.1(git).py > ...
1 #generate a code to find Armstrong number checker in python give user defined
2 def is_armstrong_number(num):
3     # Convert the number to string to easily iterate over digits
4     str_num = str(num)
5     num_digits = len(str_num)
6
7     # Calculate the sum of each digit raised to the power of num_digits
8     sum_of_powers = sum(int(digit) ** num_digits for digit in str_num)
9
10    # An Armstrong number is equal to the sum of its own digits each raised to the power of num_digits
11    return sum_of_powers == num
12
13    # Get user input
14    user_input = int(input("Enter a number to check if it is an Armstrong number: "))
15    if is_armstrong_number(user_input):
16        print(f"{user_input} is an Armstrong number.")
17    else:
18        print(f"{user_input} is not an Armstrong number.")
```

## Output:

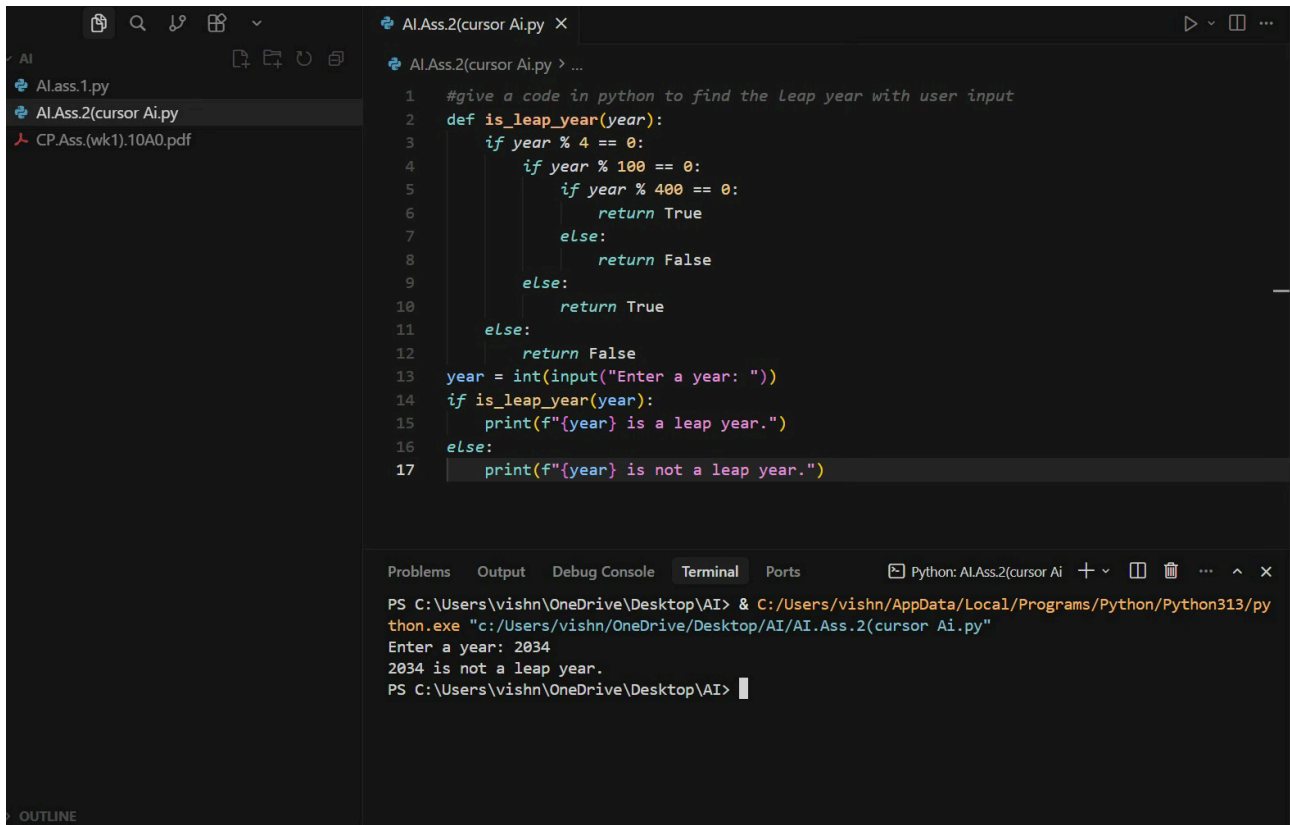
```
Enter a number to check if it is an Armstrong number: 153
153 is an Armstrong number.
```

### Task 3 :

Use Cursor AI to generate a Python program that checks whether a given year is a leap year.

Use at least two different prompts and observe changes in code.

#### Prompt:1



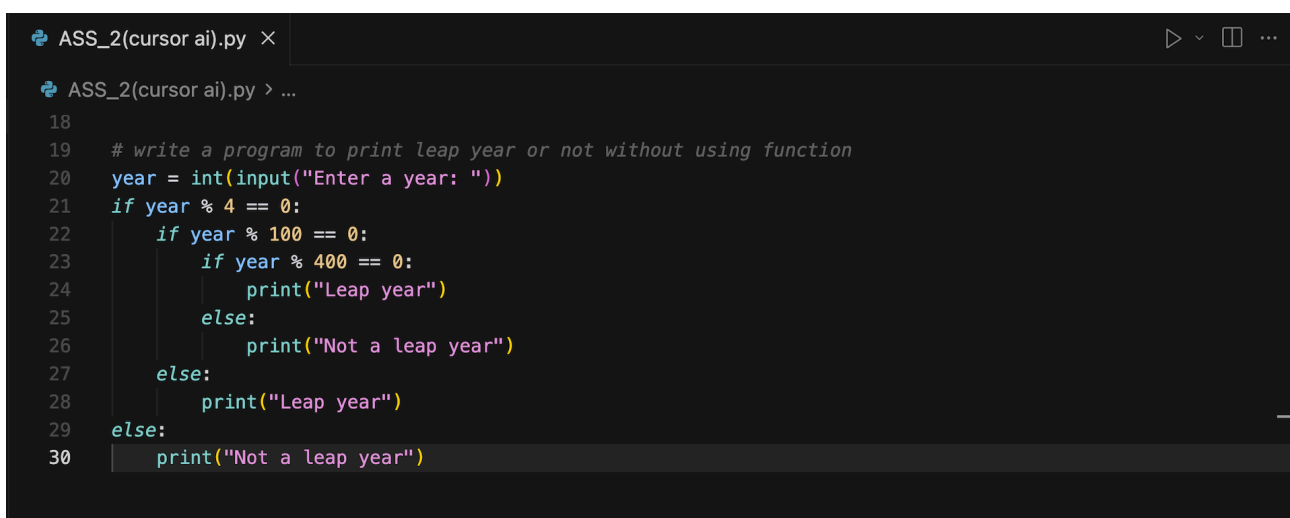
The screenshot shows the Cursor AI editor with a file named `AI.Ass.2(cursor Ai.py)`. The code is a Python program that checks if a year is a leap year. It uses a function `is_leap_year` that takes a year as input and returns a boolean. The program prompts the user to enter a year and prints the result.

```
1 #give a code in python to find the Leap year with user input
2 def is_leap_year(year):
3     if year % 4 == 0:
4         if year % 100 == 0:
5             if year % 400 == 0:
6                 return True
7             else:
8                 return False
9         else:
10            return True
11    else:
12        return False
13 year = int(input("Enter a year: "))
14 if is_leap_year(year):
15     print(f"{year} is a leap year.")
16 else:
17     print(f"{year} is not a leap year.")
```

The terminal output shows the program being executed. The user enters 2034, and the program outputs "2034 is not a leap year."

```
PS C:\Users\vishn\OneDrive\Desktop\AI> & C:/Users/vishn/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/vishn/OneDrive/Desktop/AI/AI.Ass.2(cursor Ai.py"
Enter a year: 2034
2034 is not a leap year.
PS C:\Users\vishn\OneDrive\Desktop\AI>
```

#### Prompt 2:



The screenshot shows the Cursor AI editor with a file named `ASS_2(cursor ai).py`. The code is a Python program that checks if a year is a leap year without using a function. It prompts the user to enter a year and prints the result.

```
18
19 # write a program to print leap year or not without using function
20 year = int(input("Enter a year: "))
21 if year % 4 == 0:
22     if year % 100 == 0:
23         if year % 400 == 0:
24             print("Leap year")
25         else:
26             print("Not a leap year")
27     else:
28         print("Leap year")
29 else:
30     print("Not a leap year")
```

#### Output:



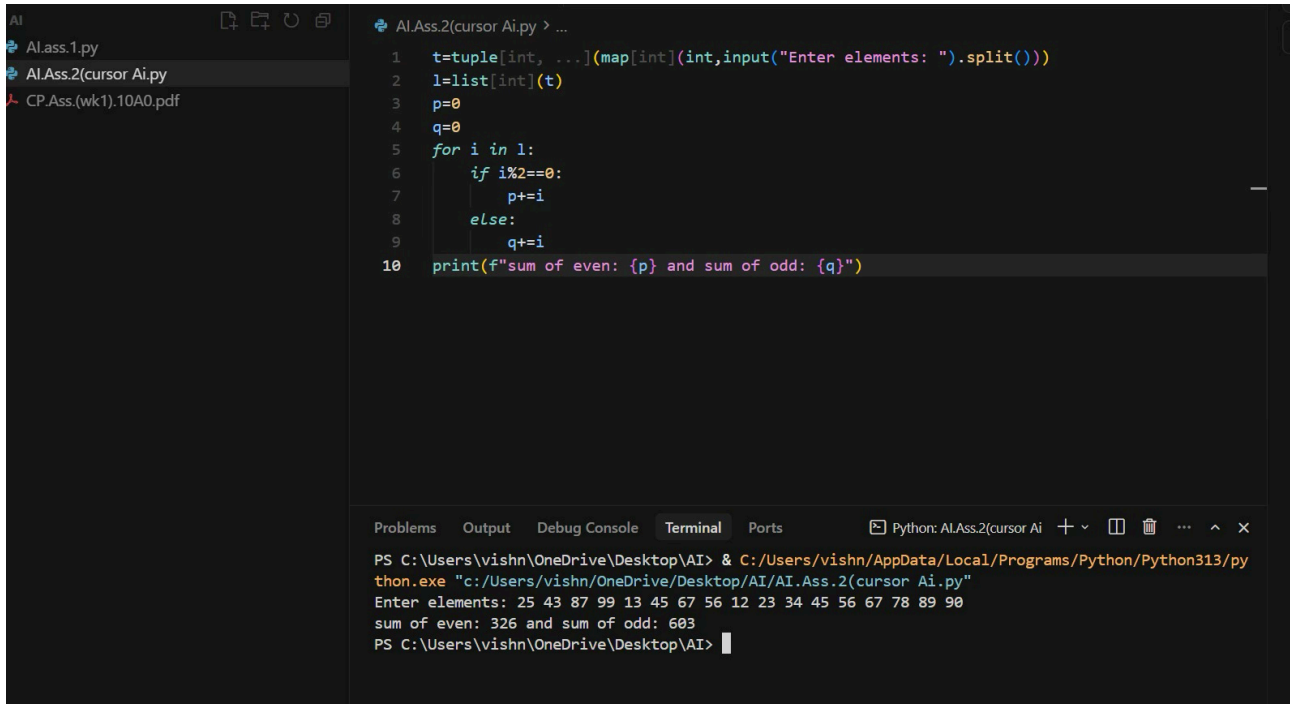
The terminal output shows the program being executed. The user enters 2024, and the program outputs "Leap year".

```
2(cursor ai).py"
Enter a year: 2024
Leap year
```

#### Task 4 :

Write a Python program that calculates the sum of odd and even numbers in a tuple, then refactor it using any AI tool.

#### Student Approach:



```
1 t=tuple[int, ...](map[int](int,input("Enter elements: ").split()))
2 l=list[int](t)
3 p=0
4 q=0
5 for i in l:
6     if i%2==0:
7         p+=i
8     else:
9         q+=i
10 print(f"sum of even: {p} and sum of odd: {q}")
```

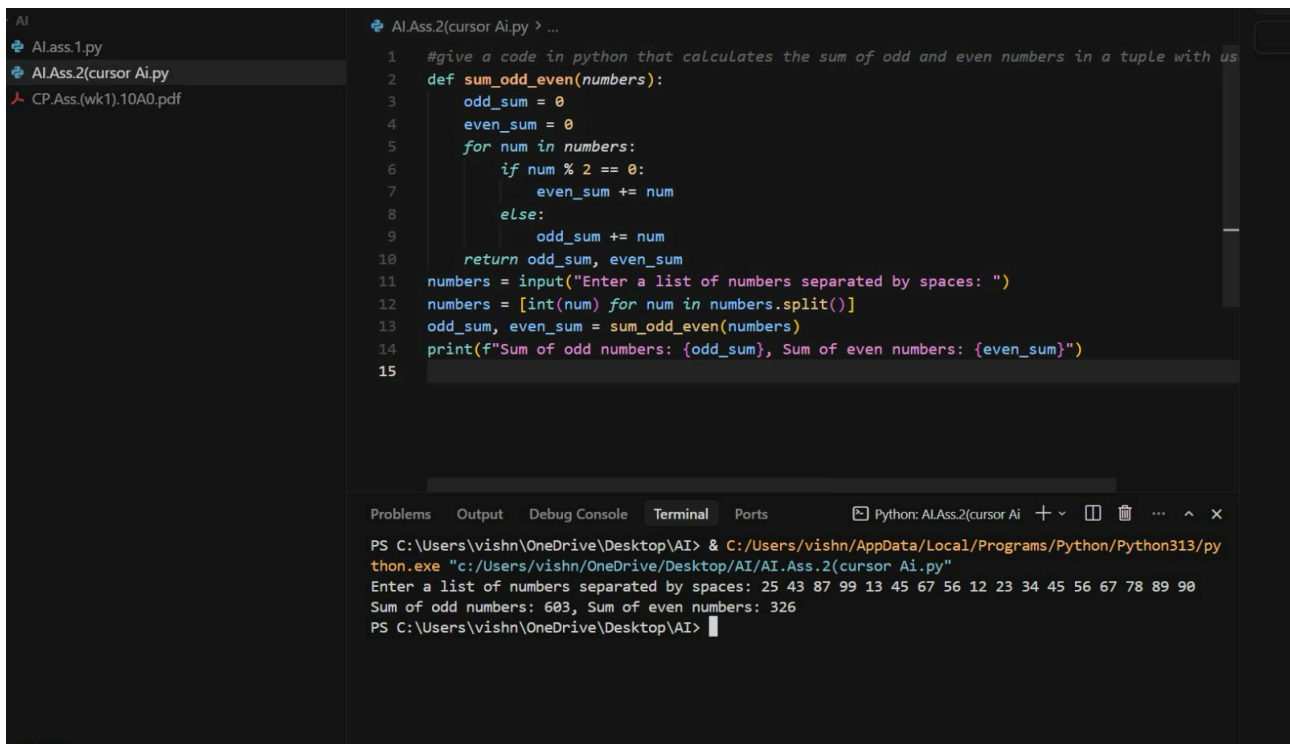
PS C:\Users\vishn\OneDrive\Desktop\AI> & C:/Users/vishn/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/vishn/OneDrive/Desktop/AI/AI.Ass.2(cursor Ai.py"

Enter elements: 25 43 87 99 13 45 67 56 12 23 34 45 56 67 78 89 90

sum of even: 326 and sum of odd: 603

PS C:\Users\vishn\OneDrive\Desktop\AI>

#### Approach of Cursor AI:



```
1 #give a code in python that calculates the sum of odd and even numbers in a tuple with us
2 def sum_odd_even(numbers):
3     odd_sum = 0
4     even_sum = 0
5     for num in numbers:
6         if num % 2 == 0:
7             even_sum += num
8         else:
9             odd_sum += num
10    return odd_sum, even_sum
11 numbers = input("Enter a list of numbers separated by spaces: ")
12 numbers = [int(num) for num in numbers.split()]
13 odd_sum, even_sum = sum_odd_even(numbers)
14 print(f"Sum of odd numbers: {odd_sum}, Sum of even numbers: {even_sum}")
15
```

PS C:\Users\vishn\OneDrive\Desktop\AI> & C:/Users/vishn/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/vishn/OneDrive/Desktop/AI/AI.Ass.2(cursor Ai.py"

Enter a list of numbers separated by spaces: 25 43 87 99 13 45 67 56 12 23 34 45 56 67 78 89 90

Sum of odd numbers: 603, Sum of even numbers: 326

PS C:\Users\vishn\OneDrive\Desktop\AI>